INDEX

II / IT / REC.

NAME: Vaarun S. STD.: SEC.: ROLL NO.: SUB.: DAA (ODS)

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		Finding the time complexity of Algorithms using Counter method:		
1)	6/8/26	problem 1	2	R
2)	6/8/26	problem 2	4	R
3)	6/8/26	problem 3	6	R
4)	6/8/26	problem 4	8	R
5)	6/8/26.	problem 5	10	R
		Divide and Conquer method		
1)	20/8/26	number of zeroes in a given array	12	R
2)	20/8/26	majority element	15	R
3)	20/8/26	Finding floor value	17	R
4)	20/8/26	Two elements sum to x	19	R
5)	20/8/26	Implementation of quick sort	22	R
		Greedy Algorithm:		
1	27/8/26	G Co in problem	25	R
2	27/8/26	G coopies problem	26	R
3.	27/8/26	G Burger problem	28	R
4.	27/8/26	G-Array sum max problem	30	R
5.	27/8/26	G product of array elements - minimum	32	R

GKC11125601 A

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		Dynamic Programming		
1		1-DP-playing withnumbers	34	
2		2-DP-Playingwith chessboard	35	
3		3-DP-Longestcommon subsequence	38	
4		4-DP-Longestnon-decreasing subsequence	40	
		Competine Programming		
1.		finding duplicate(O(n²), O(1)	42	
2.		finding duplicate(O(n) O(1).	44	
3.		Intersection of two array(O(m+n) O(1)	45	
4.		Intersection of two array(O(m+n), O(1))	49	
5.		Pair with Difference (O(n²), O(1)	51	
6.		Pair with Difference (O(n), O(1))	53	

Completed

6/8/26

Finding Time Complexity of Algorithms

Problem 1:

Aim: To convert the following algorithm into a program and find its time complexity using the counter method.

Algorithm:

```
Void function (int n){
    int i=1;
    int s=1;
    while(s<=n){
      i++;
      s+=i;
    }
}
```

Program:

```
#include <stdio.h>
Void function(int n){
    int i = 1;
    int s= 1;
    int count = 1;
    count++;
    count++;
    while (s<=n){
      count++;
      i++;
      count++;
      s+= i;
      count++;
```

```
    }
    printf("%d", count);
}
int main() {
    int n;
    scanf("%d", &n);
    function(n);
}
```

Sample Test Cases:

Input	Output
9	12
4	9

Result:
The program was written and executed successfully.

6/8/25

Problem 2:

Aim: Convert the following algorithm into program and find its time complexity using counter method.

Algorithm:

```
Void func(int n){
    if(n==1){
      printf("*");
    }
    else{
      for(int i=1; i<=n; i++){
        for(int j=1; j<=n; j++){
          printf("*");
          printf("*");
          break;
        }
      }
    }
}
```

Program:

```
#include <stdio.h>
int main(){
    int count=0;
    int n;
    scanf("%d", &n);
    if(n==1){
      count++;
      //printf("*");
      count++;
```

```
    }
  }
  else{
   count++;
   for(int i=1; count++; i<=n; count++, i++){
     for(int j=1; count++, j<=n; count++, j++){
        //printf("*");
        count++;
        //printf("*");
        count++;
        break;
      count++;
     }
   }
  }
  printf("%d", count);
}
```

Sample Test Cases:

Input	Output
2	12
1000	8002
143	717

Result:
The program was written and executed successfully.

6/8/26

Problem 8:

Aim: To convert the Algorithm into a program and find its time complexity.

Algorithm:

```
factor(int num){
  for(int i=1; i<=num; ++i){
    if(num%i==0){
      printf("%d", i);
    }
  }
}
```

Program:

```
#include <stdio.h>
int main(){
  int n;
  scanf("%d", &n);
  int count=0;
  for(int i=1; count++, i<=n; ++i){
      count++;
    if(n%i==0){
      count++; //printf("%d ", i);
    }
  }
  printf("%d", count);
}
```

Sample Test Cases:

Input	Output
12	31
25	84
4	12.

Result:
The program was written and executed sucessfully.

6/8/26

Problem 4:

Aim: To convert the algorithm into a program and find its time complexity using counter method.

Algorithm:

```
Void function (int n){
  int c=0;
  for(int i=n/2; i<n; i++){
    for (int j=1; j<n; j=2*i){
      for (int k=1; k<n; k=k*2){
          c++;
      }
    }
  }
}
```

Program:

```
#include <stdio.h>
int main(){
  int n;
  int c1=1;
  scanf("%d",&n);
  int c=0;
  for(int i=n/2; c1++,i<n; i++){
    for(int j=1; c1++,*j<n; j=2*j){
      for(int k=1; c1++,*jk<n; k=k*2){
          ~~count++;~~
          c1++;
          c++;
```

```
    }
  }
}
printf("%d", c);
}
```

Sample Test cases :

Input	Output
4	30
10	212

Result :
The program was written and executed successfully.

6/8/26

Problem 6:

Aim:
To convert the following algorithm into a program and find its time complexity using counter method.

Algorithm:

```
Void reverse (int n){
    int rev=0, remainder;
    while(n!=0){
     remainder = n%10;
     rev = rev*10 + remainder;
    n/=10;
    }
    printf(rev);
}
```

Program:

```
#include<stdio.h>
int main(){
    int n;
   scanf("%d",&n);
  int count=1;
  int rev=0, remainder;
  count++;
  count++;
  while(n!=0){
    count++;
    remainder=n%10;
```

```
        count++;
        rev = rev*10 + remainder;
        count++;
        n /= 10;
        count++;
    }
    printf("%d", count);
}
```

Sample test cases:

Input	Output
12	11
1234	19

Result:

The program was written and executed sucessfully.

20/8/26

Divide and Conquer:

Problem 1: Number of zeroes in a given array

Aim: To write a program using divide and Conquer to count the number of zeroes in the given array.

Algorithm:

```
Algorithm numzero() {
  read n, array of n elements (0,
  low = 1, high = n, firstzero = -1;
  WHILE low <= high, then
    mid = (low+high)/2;
    if array[mid] == 0
      firstzero = mid
      high = mid - 1;
    else
      low = mid+1; ENDIF
  END WHILE
  IF (firstzero == -1)
    print 0
  ELSE
    print n - firstzero
  ENDIF
END
```

Program:

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int low=0, high=n-1, firstzero=-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==0){
            firstzero=mid;
            high=mid-1;
        }
        else{
            low=mid+1;
        }
    }
    if(firstzero==-1){
        printf("%d",0);
    }
    else{
        printf("%d",n-firstzero);
    }
    return 0;
}
```

Sample Test Cases:

Input	Expected	Got
5 1 1 1 1 0 0	2	2
10 1 1 1 1 1 1 1 1 1 1	0	0
8 0 0 0 0 0 0 0 0 0	8	8

Result:

The program to find number of zeroes using divide & conquer was written and executed successfully.

Problem 2 : Majority Element

Aim : To find the majority elements in a array of size n.

Algorithm :

```
Algorithm Major(arr, l, r)
IF l = r
  return Arr[l]
mid = (l+r)/2
leftmajor ← major(arr, l, m)
rightmajor ← major(arr, m+1, r)
if leftmajor = rightmajor
  return leftmajor
leftcount ← 0
rightCount ← 0
for i ← l to r
  if arr[i] = leftmajor then
    leftcount ← leftcount + 1
  if arr[i] = rightmajor then
    rightcount ← rightcount + 1.
if leftcount > rightcount
  return leftcount;
else
  return rightcount;
```

Program:

```
#include<stdio.h>
int major(int arr[], int l, int r){
    if(l==r) return arr[l];
    int m=(l+r)/2;
    int left=major(arr, l, m);
    int right=major(arr, m+1, r);
    if(left==right) return left;
    int lc=0, rc=0;
    for(int i=l; i<=r; i++){
        if(arr[i]==left) lc++;
        if(arr[i]==right) rc++;
    }
    return (lc>rc)? left: right;
}
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    printf("%d", major(arr, 0, n-1));
    return 0;
}
```

Sample Test cases:

Input	Expected	Got
3 3 2 3	3	3

Result:

Hence the program find the majority element as written and executed successfully.

Problem 3: Finding Floor value

Aim: To write a program find the floor value of a number in an array using divide and conquer.

Algorithm:

Algorithm FindFloor() {
  Read array of size n
  Read value x.
  low ← 0
  high ← n-1
  floor ← -1
  while low ≤ high do
    mid ← (low + high)/2;
    if arr[mid] = x then
      Floor ← arr[mid]
      break
    else if arr[mid] < x then
        Floor ← arr[mid]
        low ← mid+1
    else
      high ← mid-1.
  end while
  print floor.

Program:

```
#include <stdio.h>
int main(){
  int n, x;
  scanf("%d", &n);
  int arr[n];
  for(int i=0; i<n; i++){ scanf("%d", &arr[i]); }
  int low=0, high=n-1;
  int Floor=-1; scanf("%d", &x);
```

```
while(low <= high){
  int mid=(low+high)/2;
  if(arr[mid]==x){
    int Floor=arr[mid];
      break;
  else if(arr[mid]<x){
     Floor=arr[mid];
     low=mid+1;
  }
  else{
     high=mid-1;
  }
}
printf("%d", Floor);
return 0;
}
```

Sample Test Cases :

Input	Expected	got
6 1 2 8 10 12 19 5	2	2
5 10 22 85 108 129 100	85	85

Result:
Hence the program to find the floor value of the target in an sorted array was written and executed succesfully.

Problem 4: Two Elements sum to x.

Aim: To find whether if there exist two elements in a sorted array whose sum is equal to a given value of x using Divide and Conquer method.

Algorithm:
```
Algorith find twosum()$
    read n
    read array arr[0... n-1].
    Read x
   set left <- 0
   set right <- n-1.
   while left <= right Do
     sum <- arr[left] + arr[right].
     if sum = x then
       print arr[left]
       print arr[right]
       stop
     else if sum < x then
         left <- left + 1.
     else
         right <- right - 1.
   End while
   print "NO"
   Stop.
```

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int n, x;
    scanf("%d", &n);
    int *arr = malloc(n*sizeof(int));
    for(int i=0; i<n; i++) scanf("%d", &arr[i]);
    scanf("%d", &x);
    int i=0, j=n-1;
    while(i<j){
        int sum = arr[i] + arr[j];
        if (sum==x){
            printf("%d %d\n", arr[i], arr[j]);
            free(arr);
            return 0;
        }
        else if (sum<x){
            i++;
        }
        else{
            j--;
        }
    }
    printf("NO");
    free(arr);
    return 0;
}
```

Sample Test Cases:

Input	Expected	Got
4 2 4 8 18 14	4 14	4 18
5 2 4 6 8 10 100	No	No

Result:
Thus, the program successfully checks whether the sum of two elements in the array is equal to target number.

Problem 5: Implementation of Quick sort

Aim: To sort a list of elements in ascending order using the Quick sort algorithm.

Algorithm:

algorithm Quicksort (arr, low, high)

Procedure Quicksort (arr, low, high)
    If low < high do
        PivotIndex ← partition (arr, low, high)
        Quicksort (arr, low, PivotIndex-1)
        Quicksort (arr, PivotIndex, high)
    END if
End procedure.

procedure Partition (arr, low, high)
    pivot ← arr[high]
    i ← low - 1
    for j ← low to high - 1 DO
        If arr[j] ≤ pivot Then
            i ← i+1
            swap arr[i] and arr[j]
        End if
    End for
    swap arr[i+1] and arr[high].
    return i+1.
End Procedure

Start
read n
Read array elements
Quicksort (arr, 0, n-1)
Print array -

Stop.

Program:

```
#include <stdio.h>
void swap(int *a, int *b){
    int t = *a; *a = *b; *b = t;
}
int partition(int arr[], int low, int high){
    int pivot = arr[high], i = low-1;
    for(int j = low; j < high; j++){
        if(arr[j] < pivot) swap(&arr[++i]
            &arr[j]);
    swap(&arr[i+1], &arr[high]);
    return i+1;
}
void quicksort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++) scanf("%d", &arr[i]);
    quicksort(arr, 0, n-1);
    for(int i=0; i<n; i++) scanf("%d", arr[i]);
    return 0;
}
```

Sample Test Cases:

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98
10 1 56 78 90 32 56 11 10 90 11 4	1 10 11 32 56 78 90 90 11 4	1 10 11 32 56 78 90 90 11 4
12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90

Result:

Thus the program to sort the elements of array in ascending order using quick sort algorithm was written and executed successfully

(3)

Greedy Algorithms:

Problem 1: Coin Problem.

Aim: To write a program to find the minimum number of coins/notes required to make change change for give value V.

Algorithm:

Algorithm findcount(){
Start
Read the value V from user
initialize denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1}.
initialize count = 0.
for d <- 0 to 8 do:
count <- V/denominations[d]
V <- V mod denominations[d]
print count
stop
}.

program:

```
#include <stdio.h>
int main(){
    int V;
    scanf("%d", &V);
    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;
    for (int i = 0; i < 9; i++){
        count += V/denominations[i];
        V %= denominations[i];
    }
    printf("%d", count);
    return 0;
```

}

Sample Test Cases:

Input	Expected	Got
4 9	5	5

Result:
Thus the program was written and executed successfully.

2. Problem 2: Cookies Problem:
Aim: to write a program to maximize the number of content children, given each child's greed factor and cookie's size.

Algorithm C):
Algorithm cookies(){
start.
Read n and greed factor array g[0...n-1]
Read m and cookie size array s[0...m-1];
initialize used[i]=0 for all j
initialize count=0.
for i<-0 to n-1 Do:
for j<-0 to m-1 Do:
If used[j]==0 and s[j]>=g[i]:
count <- count+1
used[j]=1
break

print count
stop
}
Program:
```
#include <stdio.h>
int main(){
    int n, m;
    scanf("%d", &n);
    int g[n];
    for(int i=0; i<n; i++) scanf("%d", &g[i]);
    scanf("%d", &m);
    int s[m], used[m];
    for(int i=0; i<m; i++){scanf("%d", &s[i]);
        use[i] = 0;
    }
    int count=0
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            if(used[j] == 0 && s[j] >= g[i]){
                count++;
                used[j] = 1;
                break;
            }
        }
    }
    printf("%d", count);
    return 0;
},
```

Sample Test cases:

Input	Expected	Got
2 1 2 3 1 2 3	2	2

Result:
Hence the program was written and executed successfully.

3. Problem 3: Burger Problem.

Aim: To write a program to determine the min. distance a person needs to run after eating a number burgers.

Algorithm:
Algorithm minCal() {
start
Read n and calorie array arr[0..n-1]
sort the array in descending order.
Initialize sum ← 0
For i ← 0 to i ← n-1 Do
sum ← sum + (n^i * arr[i]).
print sum.
Stop
}.

Program:

```
#include<stdio.h>#
int main(){
  int n;
#include<math.h>
int main(){
   int n;
   scanf("%d", &n);
   int arr[n];
   for(int i=0; i<n; i++) scanf("%d", &arr[i]
   int temp;
   for(int i=0; i<n; i++){
      for(int j=i+1; j<n; j++){
         if (arr[i] < arr[j]){
            temp=arr[i];
            arr[i] =arr[j];
            arr[j] = temp;
         }
      }
   }
   int sum=0
   for(int i=0; i<n; i++) {
      sum += (pow(n,i)*arr[i]);
   }
   printf("%d", sum);
   return 0;
}
```

Sample Test Cases:

Input	Expected	got
3 1 3 2	18	18
4 7 4 9 6	389	389
3 5 10 7	76	76

Result:
Thus the program was written and executed succesfully.

Problem 4: Array Sum Max Problem

Aim: To write a program to maximize the sum of $a[i]*i$ and its corresponding index

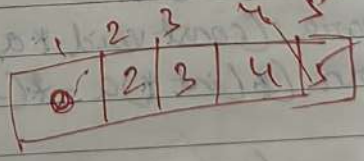
Algorithm:
Algorithm maxsum(){
start
read n and array a[0...n-1]
sort a[] in ascending order using qsort.
Initialize sum ← 0
For i ← 0 to n-1 do
sum ← sum + (a[i]*i)
print sum
stop
}

Program:

```
#include<stdio.h>
#include<stdlib.h>
int cmp(const void *a, const void *b){
    return *(int*)a - *(int*)b;
}
int main(){
    int n;
    scanf("%d", &n);
    int a[n];
    for(int i=0; i<n; i++) scanf("%d", &a[i]);
    qsort(a, n, sizeof(int), cmp);
    int sum=0;
    for(int i=0; i<n; i++) sum += a[i]*i;
    printf("%d", sum);
    return 0;
}
```

Sample Test cases:

Input	Expected	Got
5 2 5 3 4 0	40	40
2 45 3	45	45



Result:
Thus the program was written and executed successfully.

Problem 6: Product of Array Elements - Min...

Aim: To write a program to arrange two array such that sum of pairwise products is minimized.

Algorithm:

Algorithm minsum(){
start
read n, array A[0..n-1] and B[0..n-1]
sort A in ascending order
sort B in descending
Initialize sum = 0
for i = 0 to n-1 do
sum ← sum + (A[i] * B[i])
print sum
stop
}.

Program:

```
#include<stdio.h>
#include<stdlib.h>
int compare1(const void *a, const void *b){
    return (*(int *)a - *(int*)b);
}
int compare2(const void* a, const void* b){
    return (*(int *)b - *(int *)a);
}.
int main(){
    int n; sum=0;
    scanf("%d", &n);
    int a[n], b[n];
for(int i=0; i<n; i++) scanf("%d", &a[i]);
for(int i=0; i<n; i++) scanf("%d", &b[i]);
```

```
qsort(*a, n, sizeof(int), compare1);
qsort(b, n, sizeof(int), compare2);
for(int i = 0; i<n; i++){
    sum += (a[i] * b[i]);
}
printf("%d", sum);
return 0;
}
```

Sample Test Cases :

Input	Expected	Got
3 1 2 3 4 5 6	28	28
4 7 5 1 2 1 3 4 1	590	590

Result:
Thus the program was written and executed succesfully.

Dynamic Programming:
Question 1: Playing with Numbers.
Aim: To find the total number of ways a number can be represent by using 1 or 3.

Algorithm:

```
Algorithm numbers() {
  Start
  Read n
  set dp[0] = 1
  FOR i <= 1 to n
    dp[i] = dp[i-1]
    If i >= 3
      dp[i] = dp[i] + dp[i-3]
    End IF
  END FOR
  print dp[n].
  Stop
}
```

Program:

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    long long dp[n+1];
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i-1];
        if (i >= 3)
            dp[i] += dp[i-3];
    }
    printf("%d", dp[n]);
    return 0;
}
```

Output:

Input	Expected	GOT
6	6	6
2 5	8641	8641
100	2438281959 67	2438281959672
	21629	629.

Result:

Thus the program was successfully written and executed.

2. Problem 2: Playing with chessboard.

Aim: To find the maximum monetary value path from top left cell to bottom right cell while moving only right or down.

Algorithm:

```
Algorithm maxpath(){
  Start
  Read n
  Delcare matrix a[n][n].
  FOR i<-0 to n-1 DO
     FOR j<-0 to n-1 DO
        Read a[i][j]
     END FOR.
  END FOR
  FOR j<- 1 to n-1 DO
     A[0][j]<- A[0][j] + A[0][j-1]
  END FOR.
  FOR i<- 1 to n-1 DO
    A[i][0] <- A[i][0] + A[i-1][0]
  END FOR.
```

```
FOR i ← 1 to n-1 DO
    FOR j ← 1 to n-1 DO
     IF A[i-1][j] > A[i][j-1] THEN
        A[i][j] ← A[i][j] + A[i-1][j]
     ELSE
        A[i][j] ← A[i][j] + A[i][j-1]
     END IF
    ENDFOR
ENDFOR
Print A[n-1][n-1]
END
```

Program:

```
#include <stdio.h>
int main(){
   int n;
   scanf("%d", &n);
   int a[n][n];
   for(int i=0; i<n; i++){
      for(int j=0; j<n; j++){
         scanf("%d", &a[i][j]);
      }
   }
   for(int i=1; i<n; i++){
      a[0][i] = a[0][i] + a[0][i-1];
   }
   for(int j=1; j<n; j++){
      a[j][0] = a[j][0] + a[j-1][0];
   }
   for(int i=1; i<n; i++){
      for(int j=1; j<n; j++){
         if (a[i][j] > a[i][j-1])
            a[i][j] += a[i-1][j];
```

```
        else {
            a[i][j] += a[i][j-1];
        }
    }
}
printf("%d\n", a[n-1][n-1]);
}
```

Output:

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19
3 1 3 1 1 5 1 4 2 1	12	12
4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28

Result:
Thus the program to find the highest monetary path was written and executed successfully.

3 Problem: Longest Common Subsequence

Aim: To Find the length of longest common Subsequence between two strings.

Algorithm:

Algorith Subsequence() { start
Read string s1, s2
n = length of s1
m = " " s2.
Create dp[m+1][n+1].
set first row and first column to 0.
For i = 1 to n
    FOR j = i1 to m
        IF s1[i-1] == s2[j-1] Then
            dp[i][j] = 1+dp[i-1][j-1]+1
        ELSE
            dp[i][j] = maximum (dp[i-1][j], dp[i][j-1]).
        ENDIF
    ENDFOR
    Print dp[n][m]
    Stop
}.

program:

```
#include <stdio.h>
#include <string.h>
int max(int a, int b){
    return (a>b)? a: b;
}.
int main(){
    char s1[100];
    char s2[100];
    scanf("%s", s1);
```

```
    scanf("%s", s2);
    int n = strlen(s1);
    int m = strlen(s2);
    int dp[n+1][m+1];
    for(int i=0; i<=n; i++){
        dp[i][0] = 0;
    }
    for(int j=0; j<=n; j++){
        dp[0][j] = 0;
    }
    for(int i=1; i<=n; i++){
        for(int j=1; j<=m; j++){
            if(s1[i-1] == s2[j-1]){
                dp[i][j] = 1 + dp[i-1][j-1];
            }
            else{
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    printf("%d\n", dp[n][m]);
}
```

Output	Expected	Got
aab azb	2	2
ABCD ACD	4	4

Result:
Thus the program to find the longest substring between two strings was written and executed successfully.

4. problem: DP-Longest non-decreasing Subsequence.

Aim: TO find the length of longest Non-Decreasing subsequence in a given sequence.

Algorithm:

```
Algorithm subsequence(){
  Start
  Read n
  declare array arr[n] and dp[n]
  For i<-0 to n-1 DO
     Read arr[i]
     dp[i]<-1
  End For.
  mx<-1,
  for i<-1 to n-1 DO
    for j<-0 to i-1 DO
      IF arr[j]<-arr[i] Then
        If dp[j]+1 >dp[i] Then
          dp[i]<-dp[j]+1;
        ENDIF
      ENDIF
    ENDFOR
    IF dp[i] >mx Then
      mx<-dp[i]
    EndIF
    END FOR
    Print mx
    Stop
}.
```

Program:

```
#include <stdio.h>
int main(){
  int n;
  scanf("%d",&n);
  int arr[n];
  int dp[n];
  for(int i=0; i<n; i++){
    scanf("%d",&arr[i]);
    dp[i]=1;
  }
  int max=1;
  for(int i=1; i<n; i++){
    for(int j=0; j<i-1; j++){
      if(arr[j]<=arr[i]){
        if(dp[j]+1>dp[i]){
          dp[i]=dp[j]+1;
        }
      }
    }
    if(dp[i]>max){
      max=dp[i];
    }
  }
  printf("%d",max);
}
```

Output:

Input	Expected	Got
9 -1 3 4 5 2 2 2 3	6	6
7 1 2 2 4 5 7 6	6	6

Result:
Thus the program to find the longest non-decreasing subsequence in a given sequence was written and executed successfully.

Competitivity Programming:

1. Finding Duplicates ($O(n^2)$ - Time, $O(1)$ Space)

Aim: To find the duplicate element present in an list of numbers.

Algorithm:

```
Algorithm duplicates(){
    Start
    Read n, declare arr[n]
    for i<-0 to n-1 DO
        Read arr[i]
    End For
    For i<-0 to n-1 DO
        For j<-0 to n-1 DO
            If (arr[i] == arr[j] and i != j) then
                print arr[i]
                Stop
            End if
        End for
    End For
End
}
```

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d", &n); int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            if (arr[i]==arr[j] && i!=j){
                printf("%d", arr[i]);
                break;
            }
        }
    }
}
```

Inputs:

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	4	4

Result:

Thus the program to find the duplicate element was written and executed successfully.

~~2. Find the first R~~

2. Finding Duplicate O(n)-Time,

Aim: To find the duplicate element in O(n) time.

Algorithm:

```
Algorithm duplicate(){
  Start
  Read n
  declare arr[n]
  for i<-0 to n-1 R
      Read arr[i]
  End for
  For i<-0 to n-1 DO
     For j<-0 to i DO
        if arr[i]=arr[j] then
           print arr[i]
           Stop
        End if
     End for
  End for
Stop
}
```

Program:

```
#include <stdio.h>
int main(){
  int n;
  scanf("%d", &n);
  int arr[n];
  for(int i=0; i<n; i++){ scanf("%d", &arr[i]);
```

```
for(int i=0; i<n; i++){
    for(int j=0; j<i; j++){
        if(arr[i]==arr[j] && i!=j){
            printf("%d", arr[i]);
            return 0;
        }
    }
}
return 0;
}
```

Inputs:

Input	Expected	Got
11 10 9 7 6 8 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 8 4 3	1	1.

Result:
Thus the program was written and executed successfully.

3. print Intersection of 2 sorted arrays.

Aim: To print the intersection of 2 array is O(m*n) time.

Algorithm:

```
Algorithm intersection()
  Start
  Read T
  while t>0 DO
    Read n
    Set arr[n]
    For i<-0 to n DO
      Read arr[i]
    End FOR.
    Read m
    Declare arr2[m].
    For i<-0 to m-1
      Read arr2[i]
    End for
    set i<-0, j<-0, first<-1.
    while i<n and j<m DO
      if arr[i]=arr[j] then
        If first=0 then
          Print space
        End if
        Print arr[i]
        set first<-0
        i<-i+1
        j<-j+1
      Else if arr[i]<arr[j] then
        i<-i+1
      Else
        j<-j+1
      End if
    End while
    print newline
    t<-t-1.
  End while
End
```

program:

```
#include<stdio.h>
int main(){
    int t;
  scanf("%d",&t);
  while(t>0){
    int n,m;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++) scanf("%d",&arr[i]);
    scanf("%d",&m);
    int arr2[m];
  for(int i=0;i<m;i++) scanf("%d",&arr2[i]);
  int i=0,j=0;
  int first=1;
  while(i<n&&j<m){
     if(arr[i]==arr2[j]){
        if(!first) printf(" ");
        printf("%d",arr[i]);
        first=0;
        i++;
        j++;
     }
     else if(arr[i]<arr2[j]){
        i++;
    }
    else  j++;
  printf("\n");
  t--;
}
return 0;
}
```

Test Cases:

Input	Expected	Got
1 3 10 17 8 7 6 2 7 10 15 8 7 24	10 8 7	10 8 7

Result:
Thus the program was written and executed successfully.

4. Print Intersection of 2 Sorted array.

Aim: To print the common elements in an sorted array in O(m+n) time.

Algorithm:

Start
Read T
while T > 0 do
Read n1
declare array a[n1]
for i ← 0 to n1-1 Do
Read a[i]
End for
Read n2
declare array b[n2]
for i ← 0 to n2-1 DO
Read b[i]
end for.
i ← 0, j ← 0
while (i < n1 and j < n2) DO
if a[i] = b[j] then
print. a[i] + " "
i ← i+1
j ← j+1
else if a[i] < b[i] then
i ← i+1
else
j ← j+1
End if
End while
print newline
T ← T-1
End while
stop

Program:

```
#include <stdio.h>
int main(){
    int t;
    scanf("%d",&t);
    while(t--){
        int n1,n2;
        scanf("%d",&n1);
        int a[n1];
        for(int i=0;i<n1;i++) scanf("%d",&a[i]
        scanf("%d",&n2);
        int b[n2];
        for(int i=0;i<n2;i++) scanf("%d",&b[i]
        int i=0,j=0;
        while(i<n1 && j<n2){
            if(a[i]==b[j]){
                printf("%d ",a[i]);
                i++;j++;
            }else if(a[i]<b[j]){ i++;
            else j++;
        }
        printf("\n");
    }
    return 0;
}
```

Sample Test cases:

Input	Expected	Got
1 3 1 0 1 8 5 7 6 2 7 10 15 5 7 2 4 6	10 5 7	10 5 7

Result:

Thus the program as written and executed successfully.

6. Pair with Difference : $O(n^2)$ - Time complexity.

Aim:
Given an array A of sorted integers and another non-negative int k, find if there exist 2 numbers in A such that A[i] - A[j] = k, i != j.

Algorithm:
Start
Read n
declare array a[n].
For i ← 0 to n-1 DO
    Read a[i]
End for
read k
i ← 0, j ← 0
while (i < n and j < n) DO
    if (a[i] - a[j] == k) then
        pri
    if i == j then
        j = j + 1
    if (a[i] - a[j] = k) then
        print i
        exit
    else if a[i] - a[j] < k then
        j ← j + 1
    else
        i ← i + 1
    End if
Endwhile
print 0
End.

Program:

```
#include<stdio.h>
int main(){
    int n,k;
    scanf("%d",&n);
    int a[n];
    for(int i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d",&k);
    int i=0,j=1;
    while(i<n&&j<n){
        if(i==j){
            j++;
        }
        else if(a[j]-a[i]==k){
            printf("1");return 0;
        }
        else if(a[j]-a[i]<k){
            j++;
        }else{
            i++;
        }
    }
    printf("0");
    return 0;
}
```

Test Case

Input	Expected	Got
3 1 3 5 4	1	1

Result:

The program was written and executed successfully.

6. Pair with Difference, O(n) - Time complexity.

Aim: To check if there exist 2 number in a sorted array of number where their difference is equal to target.

Algorithm:

Start
Read n
declare array a[n]
for i<-0 to n-1 Do
    Read a[i]
end for
i<-0, j<-1
while (i<n and j<n) Do
  if i=j Do
    j<-j+1
elseif (a[j]-a[i]=k) Do
    print "1"
    exit
else if (a[j]-a[i]<k) Do
  j<-j+1
else
  i<-i+1
End while
print 0
End.

Program:

```
#include<stdio.h>
int main(){
  int n,k;
  scanf("%d",&n);
  int a[n];
  for(int i=0;i<n;i++) scanf("%d",&a[i]);
  scanf("%d",&k);
  int i=0,j=1;
  while(i<n&&j<n){
    if(i==j){
      j++;
    else if(a[j]-a[i]==k){
      printf("1");
      return 0;
    }else if(a[j]-a[i]<k) j++;
    else i++;
  }
  printf("0");
  return 0;
}
```

Test case:

Input	Expected	Got
3 1 3 5 4	1	1

Result:

The program was written and executed successfully.